SESSION 8: LOOPING STATEMENTS

SESSION OVERVIEW:

In this session, the students will be able to:

- Understand what looping statements are
- Understand what a for loop is
- Understand what a while loop is

KEY POINTS AND EXAMPLES:

Understand what looping statements are

Looping statements in Python are used to execute a block of code repeatedly until a specific condition is met.
Imagine you are a baker, and you need to prepare a batch of cookies. The process of making cookies involves several steps, such as mixing the ingredients, forming the dough into small balls, and baking them in the oven.

Without loops:
If you didn't have loops, you would have to repeat each step manually for every single cookie. For example, you would mix the ingredients, form a dough ball, bake it, and then start the process all over again for the next cookie. This would be extremely tedious and time-consuming, especially if you need to make a large batch of cookies.

With loops:
Looping statements in Python allow you to automate and repeat the process of making cookies without having to manually repeat each step for every single cookie. It's like having a helpful assistant that can follow your instructions repeatedly until all the cookies are made.

Python provides two types of loops:

1. for loops
2. while loops.

Understand what a for loop is

The for loop in Python is used to iterate over a sequence (such as a list, tuple, string, or range) or any other iterable object.

Think of a for loop in Python like a chef going through a list of ingredients. The list of ingredients represents the sequence or iterable object, and each ingredient is an item in the sequence. The chef, acting as the for loop, takes each ingredient one by one and uses it to prepare a dish. Similarly, the for loop iterates over each item in the sequence, allowing you to perform actions or operations on each item as needed, just like the chef prepares each ingredient for cooking.

With the for loop we can execute a set of statements, once for each item in a list, tuple, set etc.

Syntax:

```
for item in sequence:
    # code block
    # statements
```

- The for loop iterates over the items in the sequence.
- For each iteration, the current item from the sequence is assigned to the loop variable.
- The code block (indented statements) is executed for each iteration.

1) Iterating over a list/sequence:

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

The output will be:

```
apple
banana
cherry
```

In this example, the for loop iterates over each item in the fruits list, and the loop variable fruit takes on the value of each item in each iteration.

The for loop does not require an indexing variable to set beforehand.

```
# Printing even and odd numbers in a list
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

for num in numbers:
    if num % 2 == 0:
```

```
        print(f"{num} is even")
    else:
        print(f"{num} is odd")
```

The output will be:

```
1 is odd
2 is even
3 is odd
4 is even
5 is odd
6 is even
7 is odd
8 is even
9 is odd
10 is even
```

In this example:

- We have a list of numbers from 1 to 10.
- The for loop iterates over each number in the list.
- Inside the loop, an if-else statement checks if the current number is even or odd.
- If the number is even (i.e., num % 2 == 0), it prints "{num} is even".
- If the number is odd, it prints "{num} is odd".

```
# Initialize an empty list
my_list = []

# List of items to add to the list
items = [1, 2, 3, 4, 5]

# Loop over the list and append each item to the list
for item in items:
    my_list.append(item)

# Print the list to see the added items
print(my_list)
```

The output will be:

```
[1, 2, 3, 4, 5]
```

In this example:

- my_list = [] initializes an empty list.
- items = [1, 2, 3, 4, 5] is a list containing items to be added to the list.
- The for loop iterates over each item in the items list.
- my_list.append(item) appends each item from the list to the list.
- print(my_list) prints the list, showing all the added items.

```
# Initialize an empty set
my_set = set()

# List of items to add to the set
items = [1, 2, 3, 4, 5]

# Loop over the list and add each item to the set
for item in items:
    my_set.add(item)

# Print the set to see the added items
print(my_set)
```

The output will be:

```
{1, 2, 3, 4, 5}
```

In this example:

- my_set = set() initializes an empty set.
- items = [1, 2, 3, 4, 5] is a list containing items to be added to the set.
- The for loop iterates over each item in the items list.
- my_set.add(item) adds each item from the list to the set.
- print(my_set) prints the set, showing all the added items.

To find the largest element in a list without using Python's built-in functions we can use loop.

```
numbers = [10, 20, 30, 40, 3]

max_element = numbers[0]  # Assume the first element is the
maximum

for num in numbers:
    if num > max_element:
```

```
        max_element = num

print("Largest element:", max_element)
```

The output will be:

```
Largest element: 40
```

In this example:

- numbers = [10, 20, 30, 40, 3] initializes a list of integers.
- max_element = numbers[0] assumes the first element (10) as the initial maximum.
- The for num in numbers: loop iterates through each element in numbers.
- if num > max_element: checks if the current element (num) is greater than max_element.
- If num is greater, max_element is updated.
- After looping through all elements, max_element holds the largest value (40) found in numbers.

2) **Iterating over a string:**

```
greeting = "Hello, World!"
for char in greeting:
    print(char)
```

The output will be:

```
H
e
l
l
o
,

W
o
r
l
d
!
```

```
for x in "banana":
  print(x)
```

The output will be:

```
b
a
n
a
n
a
```

To remove duplicate characters from a string, we can use loop.

```
input_string = "AABBBBCCCCDDDD"
unique_chars = ""

for char in input_string:
    if char not in unique_chars:
        unique_chars += char

result_string = unique_chars
print("String with duplicates removed:", result_string)
```

The output will be:

```
String with duplicates removed: ABCD
```

In this example:

- Initialize an empty string unique_chars to store characters without duplicates.
- Iterate through each character char in input_string.
- Check if char is not already in unique_chars.
- If not present, append char to unique_chars.
- Finally, unique_chars contains the string with duplicates removed.

3) Iterating over a range:

- To loop through a set of code a specified number of times, we can use the range() function,
- The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

```
for i in range(5):
    print(i)
```

The output will be:

```
0
1
2
3
4
```

In this example, the for loop iterates over the range of numbers from 0 to 4 (the stop value 5 is not included).

```
my_list = ['Siya', 'Tiya', 'Guru', 'Daksh', 'Riya', 'Guru']

for i in range(len(my_list)):
    print(my_list[i])
```

The output will be:

```
Siya
Tiya
Guru
Daksh
Riya
Guru
```

This code iterates through each element of the my_list using a for loop and the range(len(my_list)) construct, which generates indices from 0 to len(my_list) - 1. Inside the loop, it prints each element of my_list using the index i to access each element. Therefore, it prints each element of my_list on a new line.

The range() function can also take start and step parameters:

```
for i in range(2,5):
      print(i)
```

The output will be:

```
2
3
4
```

In this example, the for loop iterates over the range of numbers from 2 to 4 (5 is not included),and increments by 1 (by default).

```
for i in range(2,2):
     print(i)
```

In the case of range(2, 2), the start value is 2, and the stop value is also 2. However, since the stop value is exclusive, the range doesn't include 2 itself. Therefore, there are no numbers within this range, and the loop will not iterate at all. As a result, the output will be blank.

```
for i in range(5,2):
     print(i)
```

This will result in a blank output because the range() function doesn't generate any values when the start parameter is greater than the stop parameter.

In the expression range(5, 2), the start parameter is 5 and the stop parameter is 2. Since the start parameter is greater than the stop parameter, there are no values within the specified range, resulting in an empty sequence.

```
for i in range(2, 10, 2):
    print(i)
```

The output will be:

```
2
4
6
8
```

In this example, the for loop iterates over the range of numbers from 2 to 9 (10 is not included), with a step size of 2.

If you want to generate values in descending order, you should specify a negative step value.

```
for i in range(5, 2, -1):
    print(i)
```

The output will be:

```
5
4
3
```

In this case, range(5, 2, -1) generates values from 5 down to 3.

```
my_list = [1, 2, 3]
for i in range(4):  # Accessing index 3, which is out of range for
my_list
    print(my_list[i])
```

This code will raise an IndexError because the index 3 is out of range for the list my_list.

```
my_number = 5
for i in my_number:
    print(i)
```

This code will raise a TypeError because integers are not iterable objects.

```
for i in range(5):
    print(x)
```

This code will raise a NameError because the variable x has not been defined before using it in the loop.

4) **Looping with enumerate():**
   - The enumerate() function is often used with the for loop to get both the index and the value of each item in a sequence.

```
fruits = ["apple", "banana", "cherry"]
for index, fruit in enumerate(fruits):
    print(index, fruit)
```

The output will be:

```
0 apple
1 banana
2 cherry
```

In this example, the for loop iterates over the fruits list, and for each iteration, enumerate() provides both the index and the value, which are assigned to the variables index and fruit, respectively.

5) **Else in For Loop:** The else keyword in a for loop specifies a block of code to be executed when the loop is finished.

```
for x in range(6):
  print(x)
else:
  print("Finally finished!")
```

The output will be:

```
0
1
2
3
4
5
Finally finished!
```

In this example,

- The for loop iterates over the numbers from 0 to 5 (inclusive) using the range function.
- Inside the loop, each value of x is printed.
- After completing all iterations of the loop, the else block is executed, and "Finally finished!" is printed.

```
# print all the factors between 2 to n.
n = 6
for i in range(2, n):
   if n % i == 0:
      print(i,end=" ")
```

The output will be:

```
2 3
```

for loop that iterates through numbers from 2 to n - 1 (inclusive). It starts from 2 because 1 is always a factor of any number, and we're excluding 1 and n from the list of factors. Secondly, check if the current number i is a factor of n by using the modulo operator (%). If n is divisible by i without leaving a remainder, then i is a factor of n.

Shorthand for loop: In Python, there is no specific "shorthand" syntax for the for loop itself. However, there are a few ways to make the for loop more concise and compact:

1) **List Comprehension:** List comprehensions in Python combine the for loop and an optional conditional expression into a single, compact expression to create a new list.

Syntax:

```
new_list = [expression for item in iterable (optional condition)]
```

or

```
new_list = [expression1 if condition else expression2 for item in
iterable]
```

```
# Create a list of squares of numbers from 1 to 10
squares = [x ** 2 for x in range(1, 11)]
print(squares)
```

The output will be:

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

In this example, the list comprehension [x ** 2 for x in range(1, 11)] creates a new list squares by iterating over the range of numbers from 1 to 10 (inclusive of 1, but not 11) using the for loop for x in range(1, 11). For each value of x, the expression x ** 2 is evaluated, and the result is added to the new list.

```
# Create a list of even numbers from 1 to 20
even_numbers = [x for x in range(1, 21) if x % 2 == 0]
print(even_numbers)
```

The output will be:

```
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

In this example, the condition if x % 2 == 0 is used to filter the list to include only even numbers from the range 1 to 20.

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = [x if x != "banana" else "orange" for x in fruits]
print(newlist)
```

The output will be:

```
['apple', 'orange', 'cherry', 'kiwi', 'mango']
```

In this example, The list comprehension [x if x != "banana" else "orange" for x in fruits] iterates over each element (x) in the fruits list. For each element, if it's not equal to "banana", it keeps the original element (x) unchanged. However, if the element is equal to "banana", it replaces it with "orange".

Python list comprehensions can contain multiple for loops and even an optional if condition.

Syntax:

```
[expression for item1 in iterable1 for item2 in iterable2 ... for
itemN in iterableN if condition]
```

- expression is the operation or transformation to be applied to the items.
- for item1 in iterable1, for item2 in iterable2, and so on, define the nested loops over the iterables.
- if condition is an optional condition that filters the items.

```
[(x, y) for x in range(3) for y in range(2)]
```

The output will be:

```
[(0, 0), (0, 1), (1, 0), (1, 1), (2, 0), (2, 1)]
```

This list comprehension iterates over each combination of x and y from the given ranges (0 to 2 for x and 0 to 1 for y), creating tuples (x, y) for each combination. So, it generates all possible pairs of (x, y) where x ranges from 0 to 2 and y ranges from 0 to 1.

List comprehensions in Python can indeed be nested within each other.

```
nested_list = [[i*j for j in range(3)] for i in range(4)]
print(nested_list)
```

The output will be:

```
[[0, 0, 0], [0, 1, 2], [0, 2, 4], [0, 3, 6]]
```

This nested list comprehension creates a list of lists where each inner list contains the result of multiplying i by j for each j in the range from 0 to 2, and each i in the range from 0 to 3.

2) **Generator Expression:** Similar to list comprehensions, generator expressions create an iterator using the same syntax as list comprehensions but with parentheses () instead of square brackets []. This can be more memory-efficient than creating a list when dealing with large iterables.

```
# Create a generator of squares
squares_generator = (x**2 for x in range(5))
print(squares_generator)
```

The output will be:

```
<generator object <genexpr> at 0x15363439f9e0>
```

This is because squares_generator is a generator object, and when you print a generator object directly, Python displays its type and memory address.

A generator object is an iterator that generates values one by one, rather than creating the entire sequence of values at once. It is lazy, meaning it generates the next value only when requested, making it more memory-efficient than creating a list or tuple of the same values.

To see the actual values generated by the generator, you can either iterate over it using a loop or convert it to a list or tuple:

```
squares_generator = (x**2 for x in range(5))
```

```
for square in squares_generator:
    print(square)
```

The output will be:

```
0
1
4
9
16
```

```
squares_generator = (x**2 for x in range(5))
print(list(squares_generator))
print(tuple(squares_generator))
```

The output will be:

```
[0, 1, 4, 9, 16]
()
```

In this example, when you try to convert squares_generator to a tuple after converting it to a list, you get an empty tuple (). This is because a generator can be consumed only once, and after converting it to a list, the generator has no more values to generate.

```
squares_generator = (x**2 for x in range(5))
print(set(squares_generator))
```

The output will be:

```
{0, 1, 4, 9, 16}
```

In this example, we create a generator squares_generator that generates the squares of numbers from 0 to 4. We then use the set() function to convert the generator to a set. The resulting set contains the unique values generated by the generator.

To convert a generator to a dictionary, you need to provide key-value pairs. One way to do this is by using a dictionary comprehension:

```
squares_generator = ((x, x**2) for x in range(5))
```

```
print(dict(squares_generator))
```

The output will be:

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

In this example, we create a generator squares_generator that generates tuples of the form (x, x**2), where x is the key, and x**2 is the value. We then use the dict() function to convert the generator to a dictionary.

Alternatively, you can use a separate generator expression to generate the keys and values:

```
keys_generator = (x for x in range(5))
values_generator = (x**2 for x in range(5))
print(dict(zip(keys_generator, values_generator)))tuple
```

The output will be:

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

we create two separate generators: keys_generator for the keys and values_generator for the values. We then use the zip() function to combine the keys and values from the two generators, and pass the resulting iterator to the dict() function to create a dictionary.

Note: Keep in mind that when you convert a generator to a set or dictionary, the generator is consumed, and you won't be able to iterate over it again. If you need to reuse the generator, you should create a new one or store the values in a list or tuple before converting them.

```
# Converting a set with duplicate values to a dictionary
my_set = {1, 2, 3, 2, 4}
my_dict = {element: element**2 for element in my_set}
print(my_dict)
```

The output will be:

```
{1: 1, 2: 4, 3: 9, 4: 16}
```

In this example, the value 2 appears twice in the set my_set. When converting to a dictionary, only the last occurrence of 2 is used as the key, and its value (4) is assigned to that key.

Nested for loop: In Python, you can nest one for loop inside another for loop. This is useful when you need to iterate over multi-dimensional data structures, such as lists of lists, or nested loops in general.

Syntax:

```
for outer_item in outer_sequence:
    # Outer loop body

    for inner_item in inner_sequence:
        # Inner loop body
```

In this syntax:

- The outer for loop iterates over the outer_sequence.
- For each iteration of the outer loop, the inner for loop iterates over the inner_sequence.
- The inner loop completes all its iterations for the current value of the outer loop before moving to the next value of the outer loop.

The outer loop runs once for each element in its iterable, and the inner loop(s) run completely for each iteration of the outer loop.

As with all Python code blocks, the body of each loop (including any inner loops) must be indented.

```
for i in range(1, 6):  # Outer loop for rows
    for j in range(1, 11):  # Inner loop for columns
        print(i * j, end="\t")
    print()  # Move to the next line after each row
```

- The outer loop iterates over the numbers 1 to 5 (range(1, 6)). These numbers represent the rows of the multiplication table.
- For each iteration of the outer loop, the inner loop iterates over the numbers 1 to 10 (range(1, 11)). These numbers represent the columns of the multiplication table.
- Within the inner loop, the product of the current row (i) and the current column (j) is calculated using i * j.
- The result of the multiplication (i * j) is printed, followed by a tab character (\t) to separate each cell in the same row.
- This process repeats for each column in the current row.
- After printing all columns for the current row, the print() statement without any

arguments is used to move to the next line. This ensures that each row of the multiplication table is printed on a new line.

Note: When you use \t, it inserts a horizontal tab space at that position.

The output will be:

```
1     2     3     4     5     6     7     8     9     10
2     4     6     8     10    12    14    16    18    20
3     6     9     12    15    18    21    24    27    30
4     8     12    16    20    24    28    32    36    40
5     10    15    20    25    30    35    40    45    50
```

```
for i in range(6):
    for j in range(2):
        print(i, j)
```

The output will be:

```
0 0
0 1
1 0
1 1
2 0
2 1
3 0
3 1
4 0
4 1
5 0
5 1
```

For every iteration of the outer loop, the inner loop iterates twice, and the outer loop iterates six times. Consequently, a total of 12 times the print statement is used. The current value of i from the outer loop and the current value of j from the inner loop is displayed after each iteration of the inner loop.

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
for row in matrix:
```

```
    for element in row:
        print(element, end=' ')
    print()
```

The output will be:

```
1 2 3
4 5 6
7 8 9
```

In this example, the outer loop iterates over each row (sublist) in the matrix list. For each row, the inner loop iterates over the elements of that row and prints them. The end=' ' argument in the print() function ensures that the elements are printed in the same line, separated by spaces.

```
nested_list = [[1, 2], [3, 4], [5, 6]]
flattened_list = []

for inner_list in nested_list:
    for element in inner_list:
        flattened_list.append(element)

print(flattened_list)
```

The output will be:

```
[1, 2, 3, 4, 5, 6]
```

In this example, the outer loop iterates over each sublist in nested_list, and the inner loop iterates over the elements of each sublist. Each element is then appended to the flattened_list.

Note: The outer loop iterates over the sub-structures (such as lists, tuples, dictionaries, or sets) within the main structure, while the inner loop iterates over the elements or sub-elements of each sub-structure.

In Python, both the outer and inner loops can have an optional else clause. The else clause in the outer loop is executed once after the loop completes normally. The else clause in the inner loop is executed after the inner loop completes for each iteration of the outer loop.

Syntax:

```
for outer_variable in outer_sequence:
    # Outer loop statements
    for inner_variable in inner_sequence:
        # Inner loop statements
    else:
        # Inner loop's else clause
else:
    # Outer loop's else clause
```

```
# Outer loop: Iterate over numbers 1 to 2
for i in range(1, 3):
    # Inner loop: Iterate over numbers 1 to 3
    for j in range(1, 4):
        # Calculate and print the product of i and j
        product = i * j
        print(f"{i} * {j} = {product}")

    # Inner loop's else clause
    else:
        print(f"Inner loop completed for {i}")

# Outer loop's else clause
else:
    print("Outer loop completed")
```

The output will be:

```
1 * 1 = 1
1 * 2 = 2
1 * 3 = 3
Inner loop completed for 1
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
Inner loop completed for 2
Outer loop completed
```

Understand what a while loop is

While loops in Python are used to repeatedly execute a block of code as long as a specified condition is true.

They provide a flexible way to iterate over a block of code until a certain condition is met, making them ideal for scenarios where the number of iterations is unknown beforehand means that when you use a while loop, you don't have to specify in advance how many times the loop will run. Instead, the loop will continue to execute its block of code as long as the specified condition remains true. This is different from a for loop, where you typically know the exact number of iterations based on the range or sequence you provide.

Imagine you're waiting in line to buy tickets for a movie. You keep checking the time on your watch, and as long as the movie hasn't started yet (condition is true), you stay in line. Once the movie begins (condition becomes false), you stop waiting and enter the theater. Similarly, a while loop in Python continuously checks a condition before executing its code block. As long as the condition remains true, the loop keeps running, but once the condition evaluates to false, the loop exits, just like you leave the line once the movie starts.

Syntax:

```
while condition:
    # Code block to be executed repeatedly
```

How While Loops Work:

- The condition is evaluated before executing the block of code inside the loop.
- If the condition is true, the block of code is executed.
- After executing the block of code, the condition is checked again.
- If the condition is still true, the block of code is executed again.
- This process continues until the condition becomes false, at which point the loop terminates and the program continues with the next line of code after the loop.

```
# Initialization
num = 1

# While loop with condition
while num <= 5:
    # Body of the loop
    print("Current number is:", num)

    # Incrementing the variable
    num += 1

# After the loop
print("Loop finished!")
```

In this example:

- num = 1 initializes the variable num to 1.
- while num <= 5: sets the loop condition to continue as long as num is less than or equal to 5.
- num += 1 increments the value of num by 1 in each iteration of the loop.
- The loop body prints the current value of num.
- Once num becomes greater than 5, the loop terminates, and "Loop finished!" is printed.

The output will be:

```
Current number is: 1
Current number is: 2
Current number is: 3
Current number is: 4
Current number is: 5
Loop finished!
```

Note: When using a while loop, it's essential to initialize the variable before the loop begins. This initialization sets the initial value of the variable and ensures that the loop condition can be evaluated properly from the start. If you forget to initialize the variable, the loop may encounter unexpected behavior or errors. Therefore, always initialize your loop control variable before entering the while loop to ensure correct and predictable loop execution.

Incrementing or decrementing the variable is necessary in most cases when using a while loop. It ensures that the loop condition can eventually become false, allowing the loop to terminate. Without this step, the loop may continue indefinitely, resulting in an infinite loop. The program will not crash, but it may consume a lot of system resources and cause the program to become unresponsive, which can be a problem.

Depending on the logic of the code, either incrementing or decrementing the variable may be used to achieve the desired behavior.

```
# Initialize a variable
count = 0

# Loop until the count is less than 5
while count < 5:
    # Check if the count is even
    if count % 2 == 0:
        print(f"{count} is even")
    # Increment the count
```

```
    count += 1
```

The output will be:

```
0 is even
2 is even
4 is even
```

In this example, the while loop runs until the count reaches 5. Inside the loop, there's an if block that checks if the count is even. If it is, it prints a message indicating that the count is even. Finally, the count is incremented in each iteration of the loop.

```
# Initialize a  variable
countdown = 10

# Loop until the countdown reaches 0
while countdown >= 0:
    print(countdown)
    # Decrement the countdown variable
    countdown -= 1
```

The output will be:

```
10
9
8
7
6
5
4
3
2
1
0
```

In this example,

- We initialize the countdown variable to 10, indicating the starting value.
- The while loop continues to execute as long as countdown is greater than or equal to 0.
- Within each iteration of the loop, we print the current value of countdown.
- After printing, we decrement the value of countdown by 1 using the -= 1 operator.

- The loop continues until countdown reaches 0, at which point it terminates.

```
# Initialize the decrement variable
num = 5

# Loop until num becomes 0
while num >= 0:
    print(num)
    # Increment num by 1
    num += 1
```

The output will result in an infinite loop because num starts at 5, and in each iteration of the loop, it increments by 1 instead of decrementing. Therefore, num will never reach 0, and the loop will continue indefinitely, printing increasing numbers endlessly.

```
# Initialize a variable
count = 0

# Loop until a condition is met
while count < 5:
    print("Count is still less than 5")
    # No increment step
```

As a result, the loop will continue indefinitely, repeatedly printing "Count is still less than 5" without ever progressing beyond that point
.

Note: To stop an infinite loop in Python,

- Using a Break Statement: You can use a break statement inside the loop to exit the loop when a certain condition is met. (will cover later).
- Keyboard Interrupt: If the program is running in a terminal or console, you can stop it by pressing Ctrl + C. This sends a KeyboardInterrupt signal to the Python interpreter, which terminates the program.

```
i=0
while i<10:
    print(i)
i = i+1
```

The output will be an infinite loop printing 0 repeatedly because the increment statement i = i + 1 is outside the loop. To fix this, the increment statement should be inside the loop block.

```
# Incorrect loop condition
value = 10

# Loop until a condition is met
while value < 5:
    print("Value is less than 5")
    value += 1
```

Since the loop condition value < 5 is initially false (because value is already 10, which is not less than 5), the code inside the while loop will not execute at all. Therefore, there will be no output produced by this code.

```
while value < 5:
    print("Value is less than 5")
    value += 1
```

The output will be:

```
NameError: name 'value' is not defined
```

The variable value is used in the loop condition and incremented within the loop body, but it's not defined or initialized before the loop. As a result, Python raises a NameError because it doesn't know what value refers to.

```
# Define an iterable object
iterable = [1, 2, 3, 4, 5]

# Initialize an index variable
index = 0

# Iterate over the iterable using a while loop
while index < len(iterable):
    print(iterable[index])
    index += 1
```

The output will be:

```
1
2
```

```
3
4
5
```

In this example, we're using a while loop to iterate over the list iterable. The loop continues until the index variable exceeds the length of the iterable. Inside the loop, we print each element of the iterable at the current index.

```
# Sum of Even Numbers till N
n = int(input())
sum = 0
i=2
while i<=n:
    sum += i
    i += 2
print(sum)
```

- The program takes an integer input n from the user.
- sum = 0: Initializes a variable sum to store the sum of even numbers. It is initialized to 0 as the sum starts from 0.
- i = 2: Initializes a variable i to 2. This is because we want to start summing from the first even number, which is 2.
- The while loop runs as long as i is less than or equal to the input number n.
- sum += i: Adds the current value of i to the sum.
- i += 2: Increments i by 2 to move to the next even number. Since we are only interested in even numbers, we increment by 2 to skip odd numbers.
- After the loop exits, the program prints the final value of sum, which represents the sum of even numbers from 2 to n.

The else Statement: With the else statement we can run a block of code once when the condition no longer is true

```
i = 1
while i < 6:
  print(i)
  i += 1
else:
  print("i is no longer less than 6")
```

The output will be:

```
1
2
3
4
5
i is no longer less than 6
```

Shorthand while loop: In Python, there isn't a specific shorthand notation like list comprehensions for while loops. If you're looking to perform a simple task in a single line, you can use a while loop with an inline condition.

```
 # Print numbers from 1 to 5 using a while loop in a single line
num = 1
while num <= 5: print(num); num += 1
```

The output will be:

```
1
2
3
4
5
```

Nested while loop: Nested while loops in Python allow you to have one while loop inside another. This structure is useful when you need to repeat a block of code multiple times, and within each iteration of the outer loop, you need to repeat another block of code multiple times controlled by the inner loop.

The inner while loop will continue to execute until its condition becomes false, and this process repeats for each iteration of the outer while loop.

Syntax:

```
outer_condition = True
while outer_condition:
    # Outer loop statements

    inner_condition = True
    while inner_condition:
        # Inner loop statements
```

In this structure, the inner while loop is nested inside the outer while loop. Both loops have their respective termination conditions, which determine when they should stop executing. The inner loop's execution is controlled by the outer loop, as it will repeat for each iteration of the outer loop.

```
# Initialize variables
outer_count = 0

# Outer loop
while outer_count < 3:
    print("Outer loop iteration:", outer_count)

    inner_count = 0
    # Inner loop
    while inner_count < 2:
        print("Inner loop iteration:", inner_count)
        inner_count += 1

    # Increment outer loop counter
    outer_count += 1
```

The output will be:

```
Outer loop iteration: 0
Inner loop iteration: 0
Inner loop iteration: 1
Outer loop iteration: 1
Inner loop iteration: 0
Inner loop iteration: 1
Outer loop iteration: 2
Inner loop iteration: 0
Inner loop iteration: 1
```

- First, we initialize the outer loop counter outer_count to 0. This variable will track the number of iterations of the outer loop.
- We start a while loop with the condition outer_count < 3. This loop will execute as long as outer_count is less than 3. Each iteration of this loop represents one iteration of the outer loop.
- Inside the outer loop, we print a message indicating the current iteration of the outer loop.

- Within the outer loop, we also initialize the inner loop counter inner_count to 0. This variable will track the number of iterations of the inner loop for each iteration of the outer loop.
- We start another while loop inside the outer loop, with the condition inner_count < 2. This inner loop will execute as long as inner_count is less than 2. Each iteration of this loop represents one iteration of the inner loop.
- Inside the inner loop, we print a message indicating the current iteration of the inner loop.
- After each iteration of the inner loop, we increment the value of inner_count by 1.
- After the inner loop completes all its iterations, we increment the value of outer_count by 1. This prepares the outer loop for the next iteration, if applicable.
- Completing Execution: Once the condition of the outer loop (outer_count < 3) becomes false, the execution of the nested while loops terminates.
- The output consists of the printed messages indicating the iteration counts for both the outer and inner loops.